

Roteiro do projeto

1. Criar o repositório no GitHub

2. Organizar a equipe

Cada grupo deve definir pelo menos:

- um responsável pelo repositório;
- alunos responsáveis pelas diferentes funcionalidades;
- quem fará as revisões dos Pull Requests.

Importante: todos os integrantes devem trabalhar em suas próprias branches.

A branch main representa o código que está integrado e funcionando.

3. Clonar o projeto

Cada integrante deve clonar o repositório em seu computador:

```
git clone URL_DO_REPOSITORIO
```

Depois:

Abrir a pasta do projeto no VSCode.

4. Criar um ambiente virtual

À esquerda no VSCode -> Python -> Environment Managers -> venv -> +

Depois defina o ambiente como padrão para o projeto.

Depois instale a dependência:

```
pytest
```

5. Executar o projeto antes de modificar

Antes de começar qualquer implementação, execute o src/main.py

Neste momento o jogo ainda terá várias partes incompletas.

Isso é esperado.

O objetivo dessa etapa é conhecer o projeto antes de começar a alterá-lo.

Também execute os testes:

pytest

ou:

python -m pytest

O teste inicial deverá funcionar.

6. Escolher uma Issue

As issues indicadas são:

#1 Implementar receber_dano()

#2 Implementar ataque do Guerreiro

#3 Implementar ataque do Inimigo

#4 Implementar ataque do Mago

#5 Implementar magia do Mago

#6 Criar Poção de Vida

#7 Implementar uso de itens

#8 Implementar turno do inimigo

#9 Implementar condição de vitória

Cada aluno deve escolher uma Issue.

A regra será:

Uma Issue → uma funcionalidade → uma branch → um Pull Request.

7. Atualizar a main

Antes de criar sua branch, atualize o projeto.

8. Criar uma branch para a Issue

No VSCode, clique em **main** no canto esquerdo inferior. Então, no topo do VSCode, clique em “Create new Branch” e atribua um nome.

9. Implementar a funcionalidade

Agora você pode modificar o código.

Antes de implementar, pense:

- O que o método deve receber?
- O que ele deve modificar?
- Qual classe é responsável por essa operação?
- O comportamento funciona para outros personagens?
- Existe algum caso especial?
- Como vou testar isso?

A ideia é utilizar os conceitos de POO estudados na disciplina.

10. Testar a funcionalidade

Sempre que possível, a implementação de uma funcionalidade deve vir acompanhada de um teste. O objetivo não é apenas verificar se o jogo funciona manualmente. Queremos que o computador consiga verificar automaticamente se o comportamento continua funcionando. Depois de implementar (métodos e respectivos testes), execute os testes:

```
pytest
```

Você também deve executar o jogo:

```
python src/main.py
```

11. Verificar o que foi alterado

No VSCode, verificar o **Source Control**.

Pergunte pertinentes a si mesmo:

- Alterei somente os arquivos necessários?
- Criei código desnecessário?
- Deixei algum TODO relacionado à minha Issue?

- Coloquei algum print() de teste que deveria ser removido?
- Os testes continuam passando?

12. Fazer o commit

Em **Changes**, selecione os arquivos a serem adicionados ao commit.

Depois realize o commit com uma mensagem explicativa. Exemplos:

feat: implementa ataque do guerreiro

fix: correção de bug

test: criação ou alteração de testes

docs: documentação

13. Enviar a branch para o GitHub

Utilize o botão de sincronizar no canto inferior esquerdo.

14. Criar o Pull Request

Entre no repositório no GitHub.

O GitHub normalmente mostrará uma mensagem semelhante a:

Compare & pull request

Clique nela.

Ou entre em:

Pull requests → New pull request

Configure:

base: main

compare: feature/ataque-guerreiro

Ou seja:

feature/ataque-guerreiro

↓

main

15. Escrever o Pull Request

O Pull Request deve explicar claramente o que foi feito.

Um modelo:

Modelo de Pull Request

Descrição

Implementa o ataque do Guerreiro.

Issue relacionada

Closes #2

Alterações realizadas

- Implementado o método atacar() da classe Guerreiro.
- O Guerreiro causa dano ao personagem alvo.
- Adicionado teste para verificar o ataque.

Testes

- pytest
- Execução manual do jogo
- Ataque do Guerreiro testado contra o Inimigo

Observações

A funcionalidade não altera o comportamento das demais classes.

O Closes #2 é especialmente interessante porque o GitHub pode fechar automaticamente a Issue quando o PR for integrado.

16. Escolher um revisor

O autor **não deve aprovar o próprio Pull Request.**

Outro integrante do grupo deve revisar.

O revisor deve verificar:

- o código;
- os testes;
- a organização;

- se a Issue foi realmente resolvida;
- se há problemas de POO;
- se o código segue o padrão do projeto.

17. Fazer Code Review

O revisor deve abrir a aba:

Files changed

e analisar as alterações.

Ele pode:

Aprovar

Se estiver tudo correto:

Approve

Solicitar alterações

Se encontrar problemas:

Request changes

Fazer um comentário

Caso tenha uma dúvida ou sugestão:

Comment

O objetivo não é simplesmente procurar erros.

O Code Review também serve para discutir decisões de programação.

Por exemplo:

"Seria melhor colocar o cálculo do dano em `receber_dano()` para que todos os personagens utilizem a mesma regra."

18. Se o revisor pedir alterações

O autor **não precisa criar outro Pull Request.**

Ele continua trabalhando na mesma Branch e corrige o problema.

Depois, envia novamente. O Pull Request será atualizado automaticamente.

19. Fazer o merge

Quando:

- a funcionalidade estiver funcionando;
- os testes estiverem passando;
- o Code Review estiver aprovado;

o Pull Request poderá ser integrado.

No GitHub:

Merge pull request

Depois:

Confirm merge

A funcionalidade passa então para a main.

20. Atualizar a main local

Depois do merge, todos os integrantes devem atualizar suas cópias (no sincronizar do VSCode).

21. Partir a próxima Issue

Voltar para o item 7.

Regras importantes do projeto

Regra 1 — Não trabalhar diretamente na main

Regra 2 — Uma Issue por funcionalidade

Regra 3 — Toda funcionalidade deve ter Issue

Regra 4 — Todo Pull Request deve ser revisado

Regra 5 — Testes devem passar antes do merge

Regra 6 — Pull Request pequeno é melhor